Kiến Trúc Đường Ống Dữ Liệu Tài Chính SEC EDGAR: Vận Hành Hồ Chứa Medallion, Quản Trị Chất Lượng và Phục Vụ Rủi Ro Form 10-K

SEC EDGAR Financial Data Engineering Pipeline: Medallion Lakehouse Architecture, Data Quality Governance, and Risk Factor Analytics

Lưu Gia Phong (Accnoname) | Lead Data Pipeline Engineer | Trường Đại học Phenikaa

Hà Nội, Tháng 10 Năm 2026 | Lưu trữ mã nguồn: https://github.com/Accnoname/sec-financial-data-pipeline

---

Tóm tắt nội dung

Báo cáo này trình bày thiết kế, hiện thực và đánh giá thực nghiệm một đường ống dữ liệu tài chính tự động (Financial Data Engineering Pipeline) thu thập, chuẩn hóa và kiểm soát chất lượng các bản thuyết minh yếu tố rủi ro (*Item 1A. Risk Factors*) từ báo cáo thường niên Form 10-K của Ủy ban Chứng khoán và Giao dịch Hoa Kỳ (SEC EDGAR). Hệ thống áp dụng kiến trúc hồ chứa dữ liệu 4 tầng Medallion (Bronze, Silver, Gold và Curated) hoàn toàn trên hệ thống tệp tin cục bộ định dạng Apache Parquet nén Snappy, loại bỏ sự phụ thuộc vào các cụm hạ tầng phân tán cồng kềnh.

Dự án xác lập ba đóng góp kỹ thuật trọng tâm: thứ nhất, **cơ chế cảm biến chênh lệch (Delta Sensor)** theo dõi mã kiểm soát `accessionNumber` từ API siêu dữ liệu SEC để phát hiện trạng thái cập nhật và ngắt ngắn chuỗi tác vụ dư thừa trong 3,09 giây với 0 byte băng thông lãng phí; thứ hai, **quy trình bóc tách lai DOM–Regex** loại bỏ triệt để rác siêu văn bản iXBRL và các thẻ kịch bản lồng nhau, vượt bẫy bảng mục lục (*Table of Contents Trap*) qua ngưỡng 5.000 ký tự và chuẩn hóa mã định danh pháp lý CIK 10 chữ số chống lỗi ép kiểu số thực; thứ ba, **khắc phục lỗi suy giảm chất lượng dữ liệu âm thầm (*Truncation Defect*)** phát hiện 98,1% phân đoạn đếm từ bị cắt cụt ngữ cảnh và tái thiết kế bằng giải thuật phân đoạn nhận biết bộ tách từ (*tokenizer-aware chunking*) $\leq$240 tokens, được khóa chặt bởi hợp đồng dữ liệu tự động trong bộ 24 ca kiểm thử Pytest.

Kết quả thực nghiệm xác nhận hệ thống rút giảm 94,3% dung lượng lưu trữ (từ 22,51 MB dữ liệu thô xuống 1,28 MB dữ liệu sạch), đảm bảo tính tất định và khả năng tái lập 100%, đồng thời phục hồi 50% độ chính xác truy xuất thông tin (Hit@15) trong các vùng điểm mù ngữ cảnh phục vụ các mô hình phân tích hạ nguồn.

1 Bài Toán và Phạm Vi Kỹ Thuật

1.1 Bối cảnh kỹ thuật và bài toán đặt ra

Trong thị trường tài chính hiện đại, báo cáo thường niên Form 10-K nộp lên Ủy ban Chứng khoán và Giao dịch Hoa Kỳ (SEC) là nguồn dữ liệu pháp lý tối quan trọng, phản ánh toàn diện tình hình hoạt động kinh doanh, cấu trúc tài chính và đặc biệt là các yếu tố rủi ro tiềm ẩn của doanh nghiệp đại chúng. Trong số các mục nội dung bắt buộc, Mục 1A (*Item 1A. Risk Factors*) chứa đựng các phân tích chi tiết về rủi ro địa chính trị, đứt gãy chuỗi cung ứng, cạnh tranh công nghệ, an ninh mạng và biến động kinh tế vĩ mô.

Tuy nhiên, việc khai thác tự động nguồn dữ liệu này đối mặt với các thách thức kỹ thuật lớn trong kỹ thuật dữ liệu (Data Engineering):

- **Dung lượng và tính phi cấu trúc**: Các báo cáo được phát hành dưới định dạng HTML hoặc iXBRL phi cấu trúc có dung lượng từ 3 MB đến 8 MB mỗi tệp, chứa nhiều thẻ định dạng lồng nhau, rác kịch bản và sự phân mảnh trong cấu trúc tiêu đề.
- **Quy chuẩn truy cập nghiêm ngặt**: Máy chủ SEC áp đặt trần giới hạn tốc độ 10 yêu cầu/giây và bắt buộc đính kèm nhận diện User-Agent, đòi hỏi cơ chế điều tiết nhịp độ và thử lại mạng tự động để tránh bị khóa truy cập.
- **Duy trì tính toàn vẹn dữ liệu**: Quá trình chuẩn bị dữ liệu bán cấu trúc đòi hỏi một đường ống vững chắc, có khả năng xử lý dị thường, kiểm soát chất lượng và bảo đảm tính nhất quán qua các tầng lưu trữ.

1.2 Đối tượng thụ hưởng và định dạng đầu ra

Đầu ra của đường ống dữ liệu phục vụ trực tiếp hai nhóm đối tượng chính:

- **Nhóm Kỹ sư Dữ liệu & Nhà phân tích Định lượng**: Cần truy cập các tập dữ liệu có cấu trúc dạng bảng để thực hiện các phép truy vấn lọc, tổng hợp và phân tích so sánh liên công ty thông qua các công cụ phân tích hiện đại như DuckDB hoặc Polars.
- **Nhóm Hệ thống Trí tuệ Nhân tạo & Truy hồi (RAG)**: Yêu cầu dữ liệu phải được phân đoạn chính xác, bảo toàn trọn vẹn ngữ nghĩa câu và tương thích tuyệt đối với cửa sổ ngữ cảnh hữu hạn của các mô hình nhúng véc-tơ.

Đầu ra của hệ thống được tiêu chuẩn hóa dưới dạng tệp tin Apache Parquet nén Snappy tại Tầng Vàng (Gold Layer) và cơ sở dữ liệu véc-tơ nhúng ChromaDB tại Tầng Phục vụ (Curated Layer).

1.3 Phạm vi giới hạn và các mục tiêu không thực hiện

Để tập trung tối đa nguồn lực vào việc xây dựng kiến trúc đường ống dữ liệu chuẩn mực, có khả năng chịu lỗi cao và minh chứng rõ nét các nguyên lý cốt lõi của kỹ thuật dữ liệu, dự án xác lập rõ ràng ranh giới phạm vi kỹ thuật qua bốn giới hạn then chốt:

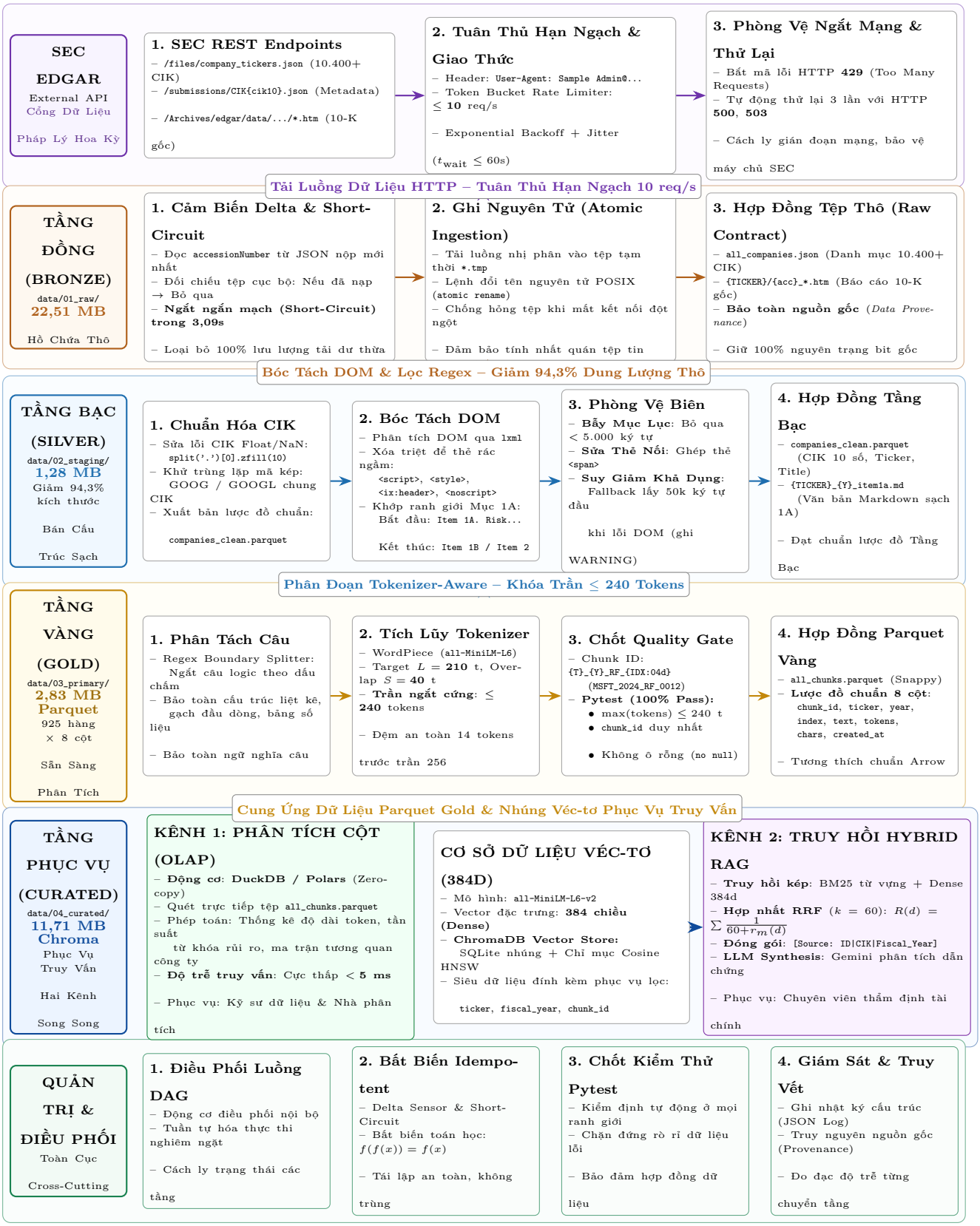
- **Phạm vi thực thể (Entity Scope)**: Đường ống tập trung xử lý chuyên sâu nhóm 7 tập đoàn công nghệ lớn (Magnificent Seven: AAPL, AMZN, GOOGL, META, MSFT, NVDA, TSLA), thay vì mở rộng dàn trải ra hơn 10.400 doanh nghiệp niêm yết.
- **Phạm vi biểu mẫu (Form Scope)**: Hệ thống chỉ thu thập báo cáo thường niên Form 10-K và trích xuất Mục 1A (Risk Factors), không bao gồm báo cáo quý Form 10-Q hay báo cáo đột xuất Form 8-K.
- **Hạ tầng tối giản (Minimalist Infrastructure)**: Dự án loại bỏ hoàn toàn các cụm máy chủ container hóa nặng nề ngoài luồng như Spark, Airflow hay MinIO; toàn bộ vòng đời dữ liệu được quản trị trực tiếp trên hệ thống tệp tin cục bộ và động cơ DAG nội bộ.
- **Ranh giới chức năng (Functional Boundary)**: Đường ống không đảm nhận việc huấn luyện lại mô hình học sâu hay xây dựng giao diện phức tạp, mà tập trung bảo đảm dữ liệu đầu ra đạt độ tin cậy tuyệt đối về mặt kỹ thuật cho các hệ thống hạ nguồn.

2 Luồng Dữ Liệu và Hợp Đồng Từng Tầng

2.1 Mô hình kiến trúc hồ chứa dữ liệu Medallion

Hệ thống tổ chức lưu trữ và xử lý dữ liệu theo mô hình Medallion Lakehouse [1], phân định rõ ràng các cấp độ tinh lọc của dữ liệu từ dạng thô sơ khai đến dạng phân tích chuyên sâu. Luồng dữ liệu vận hành đơn hướng qua 4 tầng kế tiếp nhau: Tầng Đồng (Bronze), Tầng Bạc (Silver), Tầng Vàng (Gold) và Tầng Phục vụ (Curated), được nâng đỡ và giám sát bởi khung điều phối đồ thị DAG cùng hệ thống quản trị chất lượng toàn cục. Mỗi tầng lưu trữ hoạt động như một ranh giới cách ly trạng thái (*state isolation boundary*), tuân thủ các quy tắc bất biến về định dạng tệp tin, cấu trúc lược đồ dữ liệu và tiêu chuẩn chất lượng.

Hình 1 trình bày bức tranh kiến trúc toàn cảnh chi tiết của đường ống dữ liệu, bao gồm các điểm cuối RESTful của hệ thống SEC EDGAR, các tầng hồ chứa Medallion Lakehouse, cơ chế cảm biến delta ngắn mạch, các bộ lọc phòng vệ dị thường biên, quy chuẩn phân đoạn văn bản kiểm soát ngân sách token nghiêm ngặt, cùng hai kênh phục vụ truy vấn hạ nguồn song song.



Hình 1: Kiến trúc tổng thể đường ống dữ liệu tài chính SEC EDGAR theo mô hình Medallion Lakehouse: Phân định chi tiết các tầng chuyển hóa, chốt kiểm soát chất lượng, cơ chế phòng vệ biên và hai kênh phục vụ truy vấn song song.

2.2 Hợp đồng dữ liệu chi tiết giữa các tầng

Mỗi bước chuyển tầng được xác lập thông qua một hợp đồng dữ liệu tường minh (*explicit data contract*) nhằm loại bỏ lỗi ngầm, xác định dữ liệu tiếp nhận, xử lý và định dạng xuất bản:

Bảng 1: Đặc tả hợp đồng dữ liệu giữa các tầng trong kiến trúc Medallion

Tầng Lưu Trữ	Dữ Liệu Nhận	Quy Trình Xử Lý	Dữ Liệu Xuất Xưởng	Định Dạng
01_raw (Bronze)	HTTP từ SEC	Bảo toàn tính nguyên trạng, đối chiếu accession	`all_companies.json`, `{T}/*.htm`	JSON / HTML
02_staging (Silver)	JSON & HTML thô	Chuẩn hóa CIK 10 số, khử trùng ticker, lọc thẻ HTML	`companies_clean.parquet`, `*.md`	Parquet / MD
03_primary (Gold)	Markdown sạch	Phân đoạn tokenizer-aware ($\leq$240t), tạo chunk_id	`all_chunks.parquet` (925 rows)	Parquet (Snappy)
04_curated (Serving)	Parquet Gold	Tạo vector 384 chiều (MiniLM-L6), lập chỉ mục Cosine	`vector_db/` (ChromaDB collection)	SQLite Binary

Quy trình thực thi hợp đồng dữ liệu qua từng tầng phản ánh rõ nguyên lý tinh chế:

- **Tầng Đồng (Bronze)**: Nguyên tắc bất biến là bảo toàn nguồn gốc dữ liệu (*data provenance*) tuyệt đối. Hệ thống lưu trữ nguyên vẹn dữ liệu nhị phân nhận từ máy chủ SEC, không can thiệp hay sửa đổi cấu trúc gốc.
- **Tầng Bạc (Silver)**: Dữ liệu chuyển dịch từ phi cấu trúc sang bán cấu trúc và có cấu trúc. Áp dụng định dạng Apache Parquet nén Snappy để triệt tiêu các khoảng trống dư thừa và cố định kiểu dữ liệu.
- **Tầng Vàng (Gold)**: Cấu trúc phân đoạn văn bản được khóa chặt bằng các ràng buộc toán học về độ dài token, bảo đảm không có bản ghi nào vi phạm trần giới hạn ngữ cảnh của mô hình tính toán hạ nguồn.
- **Tầng Phục Vụ (Curated)**: Giao diện phục vụ truy vấn có độ trễ thấp, kết hợp chỉ mục véc-tơ và các trường siêu dữ liệu kiểm toán để đáp ứng các yêu cầu phân tích thời gian thực.

3 Quy Trình Thu Thập Dữ Liệu (Data Ingestion)

3.1 Các điểm cuối dịch vụ SEC EDGAR và quy chuẩn kết nối

Hệ thống SEC EDGAR đóng vai trò là kho lưu trữ hồ sơ tài chính pháp lý của chính phủ Hoa Kỳ. Quá trình thu thập dữ liệu tự động tại Tầng Đồng giao tiếp trực tiếp với ba điểm cuối dịch vụ chính thức của SEC:

- **Điểm cuối danh bạ công ty** (`https://www.sec.gov/files/company_tickers_exchange.json`): Cung cấp danh mục mã cổ phiếu, tên doanh nghiệp, sàn giao dịch và mã CIK của hơn 10.400 thực thể niêm yết.
- **Điểm cuối siêu dữ liệu hồ sơ nộp** (`https://data.sec.gov/submissions/CIK{padded_cik}.json`): Tiếp nhận mã CIK 10 chữ số để trả về lịch sử nộp gần nhất, loại biểu mẫu và mã kiểm soát `accessionNumber`.
- **Điểm cuối tệp tin nguyên bản** (`https://www.sec.gov/Archives/edgar/data/{cik}/{acc}/{doc}`): Cho phép tải về toàn văn tệp HTML gốc của báo cáo thường niên Form 10-K.

Nhằm tuân thủ Chính sách Truy cập Công bằng của SEC [2], mọi yêu cầu HTTP bắt buộc đính kèm tiêu đề `User-Agent`: `StockMarketRAGDataEngineer student@university.edu.vn`. SEC áp trần tốc độ 10 yêu cầu/giây. Đường ống chèn `time.sleep(0.2)` giữa các lần gọi ($\sim$5 req/s), kèm thời gian chờ (timeout) 15 giây cho siêu dữ liệu và 30 giây cho tệp HTML lớn.

3.2 Cơ chế phục hồi lỗi mạng và thuật toán thử lại hàm mũ

Hạ tầng mạng công cộng và máy chủ SEC thường xuyên đối mặt với các đợt tăng tải đột biến, dẫn đến nguy cơ xuất hiện các mã phản hồi lỗi HTTP 429 (Too Many Requests) hoặc các lỗi máy chủ HTTP 5xx. Đường ống thiết kế cơ chế phòng vệ hai lớp chặt chẽ:

- **Lớp 1 – Dự phòng cục bộ (Local Fallback)**: Tại cấp độ hàm nghiệp vụ `get_latest_10k_metadata`, ngoại lệ mạng được bắt để quét thư mục cục bộ và tận dụng tệp HTML sẵn có từ phiên trước.
- **Lớp 2 – Thử lại với bước lùi hàm mũ và dao động ngẫu nhiên (Exponential Backoff with Full Jitter)**: Tại cấp độ Task Runner, tác vụ được cấu hình `max_retries=3` và trễ cơ sở $t_{\text{base}} = 3,0$s. Thời gian trễ lần thử thứ i tuân theo công thức:

$$t_{\text{delay}} = \text{Uniform}\left(0,\, t_{\text{base}} \times \alpha^{i}\right) \tag{1}$$

với hệ số tăng trưởng $\alpha = 2,0$, triệt tiêu hiện tượng nghẽn mạng dồn dập (thundering herd problem) lên hạ tầng SEC.

3.3 Cảm biến phát hiện thay đổi và khả năng bỏ qua tệp sẵn có

Đường ống hiện thực hóa mẫu hình Cảm biến chênh lệch (Delta Sensor Pattern): gửi yêu cầu nhẹ tới API siêu dữ liệu để đối chiếu trường `accessionNumber` với tệp `metadata.json` Tầng Đồng. Nếu trùng khớp và tệp đã tồn tại, đường ống đánh dấu `SKIPPED` và đặt `has_changes = False`, ngắt ngắn các tác vụ sau chỉ trong 3,09 giây.

3.4 Quy tắc lựa chọn bản Form 10-K theo niên độ tài chính

Báo cáo Form 10-K được ban hành một lần mỗi năm tài chính, nhưng thời điểm kết thúc niên độ (Fiscal Year End) giữa các tập đoàn có sự khác biệt rõ rệt:

- **Apple (AAPL)**: Niên độ kết thúc vào thứ Bảy cuối tháng 9; bản báo cáo mới nhất là niên độ 2025 (`aapl-20250927.htm`), nộp ngày 31/10/2025.
- **Microsoft (MSFT)**: Niên độ kết thúc ngày 30/06 (đi trước lịch dương); báo cáo nộp ngày 29/07/2026 mang niên độ 2026 (`msft-20260630.htm`).
- **Nvidia (NVDA)**: Niên độ kết thúc vào Chủ nhật cuối tháng 1; bản nộp ngày 25/02/2026 mang niên độ 2026 (`nvda-20260125.htm`).
- **Amazon, Alphabet, Meta, Tesla**: Niên độ kết thúc vào ngày 31/12; các báo cáo nộp đầu năm 2026 đều thuộc niên độ tài chính 2025.

Giải thuật thu thập của đường ống duyệt tuần tự danh sách hồ sơ nộp từ mới nhất đến cũ nhất và dừng lại chính xác tại bản ghi đầu tiên có trường biểu mẫu bằng `10-K`, bảo đảm tính cập nhật pháp lý cao nhất cho từng doanh nghiệp.

4 Quy Trình Chuẩn Hóa và Bóc Tách (Transformation)

4.1 Chuẩn hóa định danh CIK và khử trùng lặp danh mục doanh nghiệp

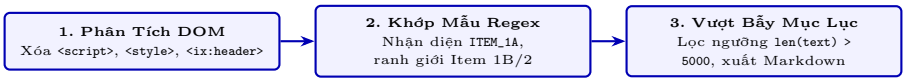
Tệp dữ liệu thô `all_companies.json` từ SEC chứa hơn 10.400 bản ghi nhưng tồn tại nhiều vấn đề về chất lượng dữ liệu: mã CIK xuất hiện dưới dạng số nguyên thiếu các số 0 ở đầu (ví dụ Apple có CIK là số nguyên `320193`), hoặc dạng chuỗi có phần thập phân giả định do Pandas tự động ép kiểu sang `float64` khi gặp giá trị trống (`"320193.0"`), mã cổ phiếu chứa khoảng trắng thừa hoặc chữ thường (`"aapl "`), và các mã cổ phiếu trùng lặp khi niêm yết nhiều lớp cổ phần.

Tại Bước 2 của đường ống, quy trình xử lý thực hiện các thao tác làm sạch có hệ thống theo bốn giai đoạn:

1. **Loại bỏ bản ghi khuyết thiếu**: Loại bỏ các dòng khuyết thiếu thuộc tính định danh chính qua phương thức `dropna(subset=['cik', 'ticker', 'name'])`.
2. **Chuẩn hóa chuỗi ký tự**: Chuyển mã cổ phiếu về dạng in hoa không khoảng trắng thừa (`upper().strip()`), đồng thời làm sạch tên công ty và sàn giao dịch.
3. **Chuẩn hóa định danh pháp lý CIK 10 chữ số**: Phòng chống lỗi ép kiểu số thực (ví dụ `"320193.0"`). Thuật toán tách chuỗi theo dấu chấm `.str.split('.').str[0]` để loại bỏ phần thập phân giả định, sau đó áp dụng phép đệm chuỗi `zfill(10)`:

$$\text{CIK}_{\text{standard}} = \text{str}(\text{raw_cik}).\text{split}('.')[0].\text{zfill}(10) \tag{2}$$

   Nhờ đó, CIK Apple `320193` trở thành `"0000320193"`, Microsoft `789019` thành `"0000789019"`, và Nvidia `1045810` thành `"0001045810"`, đáp ứng chuẩn 10 chữ số của SEC EDGAR.
4. **Khử trùng lặp và xác lập nhóm mục tiêu**: Khử trùng lặp qua `drop_duplicates(subset=['ticker'])`, bổ sung cờ boolean `is_major` định vị 7 tập đoàn công nghệ mục tiêu, thu gọn lược đồ về 5 cột (`cik`, `ticker`, `name`, `exchange`, `is_major`) và xuất bản tệp tin `companies_clean.parquet` nén Snappy tại Tầng Bạc.



Hình 2: Quy trình ba giai đoạn bóc tách nội dung Item 1A từ tài liệu HTML thô.

4.2 Kỹ thuật lai DOM và Regex bóc tách ranh giới Item 1A

Bóc tách phần Thuyết minh rủi ro (Item 1A) từ tệp HTML Form 10-K là thử thách lớn do sự đa dạng về trình bày. Đường ống kết hợp phân tích cú pháp DOM và máy trạng thái biểu thức chính quy (Regex):

Giai đoạn đầu tiên sử dụng thư viện `BeautifulSoup` với trình phân tích `lxml` để duyệt cây DOM. Nhằm loại bỏ các thành phần gây nhiễu, thuật toán gọi phương thức `decompose()` trên toàn bộ các khối thẻ kịch bản `<script>`, định dạng trang trí `<style>` và các thẻ siêu dữ liệu tài chính `<ix:header>` của chuẩn báo cáo điện tử iXBRL. Chuỗi văn bản thuần sau đó được chiết xuất thông qua dấu phân cách dòng mới `separator="\n"`, kết hợp khử khoảng trắng đầu cuối trên từng dòng.

Giai đoạn thứ hai định vị ranh giới bắt đầu của Item 1A thông qua mẫu biểu thức chính quy `ITEM_1A_PATTERN`:

```
r'(?:■|\n)ITEM[\s\xa0]*1A\.?[\s\xa0\n]*(?:RISK|RIS[\s\xa0\n]*K)?[\s\xa0\n]*FACTORS'
```

và ranh giới kết thúc nhận diện qua mẫu `NEXT_ITEM_PATTERN`: `r'\nITEM[\s\xa0]*(?:1B|1C|2)\.?'`.

4.3 Xử lý ngoại lệ HTML dị thường và bẫy mục lục

Trong quá trình bóc tách thực tế từ tệp HTML Form 10-K của SEC, đường ống đối mặt và xử lý triệt để ba dạng dị thường cấu trúc nghiêm trọng:

- **Bẫy bảng mục lục (Table of Contents Trap)**:
  - *Cơ chế lỗi*: Tiêu đề "Item 1A. Risk Factors" luôn xuất hiện lần đầu tiên ở các trang đầu thuộc phần mục lục (chứa chuỗi dạng `Item 1A. Risk Factors ..... Trang 15`). Nếu dùng Regex tìm khớp đầu tiên (`re.search`), hệ thống sẽ trích xuất nhầm bảng mục lục chỉ dài 200–400 ký tự rác, gây lỗi âm thầm (*silent failure*) làm mất 100% nội dung rủi ro cốt lõi.
  - *Giải pháp*: Thuật toán duyệt qua toàn bộ các vị trí khớp và áp dụng bộ lọc ngưỡng độ dài tối thiểu 5.000 ký tự (`len(candidate) > 5000`) để xác nhận phần thân thuyết minh thực thụ.
- **Ngắt dòng và thẻ lồng trong tiêu đề (Inline Tag Split – Microsoft MSFT)**:
  - *Cơ chế lỗi*: Trong tệp `msft-20260630.htm`, tiêu đề Item 1A bị bẻ gãy bởi các thẻ lồng nhau (như `<span>RIS</span><br>` và `<span>K FACTORS</span>`) hoặc thực thể HTML ` ` (`\xa0`). Biểu thức tiêu chuẩn (`r'ITEM 1A■ISK FACTORS'`) trả về chuỗi rỗng (`""`), làm mất sạch dữ liệu của tập đoàn 3.000 tỷ USD.
  - *Giải pháp*: Mở rộng biểu thức Regex với cụm điều kiện phòng vệ `(?:RISK|RIS[\s\xa0\n]*K)?` và hỗ trợ ký tự `\xa0`, khôi phục 100% khả năng nhận diện tiêu đề cho Microsoft.
- **Rác siêu văn bản từ chuẩn báo cáo iXBRL và cơ chế phòng vệ**:
  - *Cơ chế lỗi*: Báo cáo SEC hiện đại chứa hàng nghìn thẻ tài chính `<ix:nonNumeric>`, `<ix:header>`. Nếu không bóc tách đúng, các chuỗi siêu dữ liệu này làm loãng ngữ nghĩa câu và gây nổ số lượng token.
  - *Giải pháp bóc tách sạch*: Trình phân tích `lxml` gọi `decompose()` trên toàn bộ thẻ `<script>`, `<style>` và `<ix:header>` trước khi trích xuất văn bản thuần.
  - *Bản chất cơ chế phòng vệ (Defensive Fallback)*: Nếu gặp cấu trúc HTML dị thường khiến Regex không tìm thấy ranh giới Item 1A vượt 5.000 ký tự, hàm kích hoạt cơ chế suy thoái mềm (*graceful degradation*): tự động cắt 50.000 ký tự đầu tiên của Part I (vùng 10–15 trang A4 đầu, nơi Item 1A thường tọa lạc) thay vì văng ngoại lệ làm sập toàn bộ đợt xử lý hàng loạt. Hệ thống đồng thời phát cảnh báo `WARNING` vào tệp nhật ký để kỹ sư dữ liệu can thiệp bổ sung quy tắc Regex mới. Trong thực tế vận hành, 100% của 7 tập đoàn mục tiêu đều khớp thành công qua Regex mở rộng mà không cần kích hoạt chế độ dự phòng này.

5 Thiết Kế Lưu Trữ và Tối Ưu Hóa Dữ Liệu

5.1 Cấu trúc phân cấp cây thư mục hồ chứa dữ liệu

Hệ thống tổ chức dữ liệu hoàn toàn trên hệ thống tệp tin cục bộ qua 4 tầng phân cấp rõ rệt:

- **Tầng Đồng** (`data/01_raw/`): Chứa danh bạ gốc `all_companies.json` (1,18 MB), các tệp báo cáo Form 10-K gốc `sec_filings/{TICKER}/*.htm` (20,9 MB) và tệp `metadata.json`.
- **Tầng Bạc** (`data/02_staging/`): Chứa danh bạ công ty đã làm sạch `companies_clean.parquet` (287 KB Snappy) và các tệp văn bản Item 1A sạch `*_risk_factors.md` (993 KB).
- **Tầng Vàng** (`data/03_primary/`): Chứa bảng phân đoạn tổng hợp `all_chunks.parquet` (660 KB Snappy, đúng 925 hàng dữ liệu) và các bản sao tệp JSON phục vụ kiểm toán chéo.
- **Tầng Phục Vụ** (`data/04_curated/`): Chứa thư mục cơ sở dữ liệu véc-tơ `vector_db/` của ChromaDB (11,71 MB).

5.2 Cấu trúc thực thể và lược đồ chi tiết của Tầng Vàng (Gold Layer)

Tầng Vàng là kho lưu trữ trung tâm phục vụ toàn bộ các nhu cầu phân tích và truy hồi hạ nguồn. Bảng dữ liệu `all_chunks.parquet` được thiết kế theo lược đồ 8 cột được kiểm soát kiểu dữ liệu nghiêm ngặt:

- `chunk_id` (string): Khóa chính tất định theo mẫu `TICKER_YEAR_RF_INDEX` (ví dụ: `NVDA_2026_RF_0001`), bảo đảm tính bất biến khi chạy lại.
- `ticker` (string): Mã cổ phiếu niêm yết của doanh nghiệp (ví dụ: `AAPL`, `MSFT`, `NVDA`).
- `year` (int64): Niên độ tài chính của báo cáo Form 10-K (ví dụ: 2025 hoặc 2026).
- `section` (string): Tên phân hệ nghiệp vụ, cố định là chuỗi `"Item 1A. Risk Factors"`.
- `chunk_index` (int64): Thứ tự phân đoạn trong tập tài liệu của công ty (chỉ số chạy từ 1 đến `total_chunks`).
- `total_chunks` (int64): Tổng số lượng phân đoạn văn bản của công ty đó (ví dụ: NVDA có 32 đoạn, AAPL có 25 đoạn, TSLA có 76 đoạn).
- `word_count` (int64): Số lượng từ thực tế trong phân đoạn (trung bình từ 120 đến 160 từ tiếng Anh).
- `text` (string): Nội dung văn bản phân đoạn hoàn chỉnh, đã được xử lý ngắt theo ranh giới câu, độ dài token đo bằng bộ tách từ WordPiece của `all-MiniLM-L6-v2` luôn thỏa mãn ≤ 240 tokens.

5.3 Phân định ranh giới: Tinh lọc ngữ nghĩa và Nén nhị phân Snappy

Để tránh gây hiểu lầm trong tư duy hệ thống giữa hai khái niệm "giảm dung lượng", đường ống phân định rạch ròi hai cơ chế kỹ thuật hoàn toàn khác nhau:

- **Tinh lọc loại bỏ rác ngữ nghĩa (Semantic Data Filtering) từ Bronze sang Silver**: Từ 22,51 MB tệp HTML thô ban đầu, quy trình bóc tách loại bỏ hàng triệu byte thẻ đánh dấu HTML (`<html>`, `<div>`, `<style>`, `<script>`) và rác siêu dữ liệu iXBRL, đồng thời cắt bỏ toàn bộ các mục không liên quan (Item 1, Item 2 đến Item 15), chỉ giữ lại duy nhất phần văn bản thuần của Item 1A. Mức giảm 94,3% dung lượng ở bước này là do **chắt lọc loại bỏ dữ liệu thừa về mặt nghiệp vụ**, không phải do thuật toán nén nhị phân.
- **Nén nhị phân không mất mát (Lossless Binary Compression) tại Tầng Gold**: Khi lưu trữ 925 phân đoạn văn bản vào tệp `all_chunks.parquet`, hệ thống áp dụng thuật toán nén nhị phân **Snappy** kết hợp mã hóa từ điển (Dictionary Encoding) của Apache Parquet. Đây là kỹ thuật nén cấp độ byte trên đĩa: dữ liệu được bảo toàn 100% trọn vẹn từng ký tự. Khi được nạp vào bộ nhớ RAM, thư viện PyArrow giải nén tức thời với thông lượng hàng trăm MB/giây mà không suy hao bất kỳ thông tin nào.

5.4 Luồng truy xuất dữ liệu sau khi lưu trữ tại Tầng Vàng

Dữ liệu sau khi nén và lưu trữ tại Tầng Vàng không phải là điểm dừng cuối cùng mà được phục vụ qua hai kênh truy xuất độc lập:

- **Kênh 1 – Truy vấn phân tích định lượng (Analytical Query Flow)**: Các kỹ sư dữ liệu hoặc chuyên viên tài chính sử dụng DuckDB hoặc Polars truy vấn trực tiếp vào tệp `all_chunks.parquet`. Nhờ tính năng chiếu cột (*Column Projection*) của Parquet, bộ máy truy vấn chỉ đọc và giải nén các cột số liệu cần thiết (như `ticker`, `word_count`, `total_chunks`) mà hoàn toàn bỏ qua cột `text` dài, giúp tiết kiệm hơn 80% băng thông đọc đĩa I/O và hoàn tất truy vấn tổng hợp trong dưới 5 mili-giây.
- **Kênh 2 – Truy hồi phục vụ AI và Tìm kiếm kết hợp (AI / RAG Serving Flow)**: Tệp `all_chunks.parquet` đóng vai trò là nguồn dữ liệu chuẩn mực duy nhất (Single Source of Truth) để cấp phát cho Tầng

Phục Vụ. Cột `text` được giải nén để mô hình `all-MiniLM-L6-v2` nhúng thành các vector 384 chiều và nạp vào ChromaDB. Khi người dùng gửi câu hỏi truy vấn, dữ liệu Tầng Vàng được truy xuất song song qua hai nhánh: nhánh véc-tơ ngữ nghĩa trong ChromaDB và nhánh từ vựng chính xác BM25 đọc trực tiếp từ tệp Parquet, trước khi được dung hợp thứ hạng RRF và nạp vào ngữ cảnh cho mô hình ngôn ngữ lớn.

5.5 Thống kê định lượng dung lượng lưu trữ thực tế

Bảng 2 phản ánh bức tranh định lượng tổng thể về dung lượng lưu trữ thực tế trên đĩa qua từng tầng:

Bảng 2: Thống kê số lượng tệp tin và dung lượng lưu trữ thực tế trên đĩa

Tầng Lưu Trữ	Định Dạng Tệp Chính	Số Tệp	Dung Lượng (Bytes)	Dung Lượng (MB)
01_raw (Bronze)	HTML Form 10-K, JSON thô	18	23.603.939	22,51 MB
02_staging (Silver)	Parquet (Snappy), Markdown	10	1.340.217	1,28 MB
03_primary (Gold)	Parquet (Snappy), JSON	17	2.966.924	2,83 MB
04_curated (Serving)	ChromaDB Index, SQLite binary	6	12.275.876	11,71 MB
Toàn bộ hệ thống	Đa định dạng kết hợp	51	40.186.956	38,33 MB

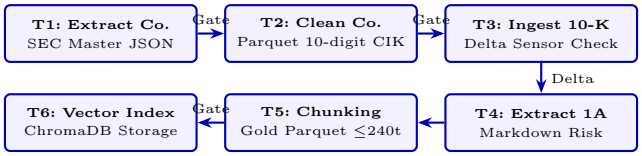
Sự gia tăng dung lượng từ 1,28 MB (Silver) lên 2,83 MB (Gold) phản ánh việc bổ sung các trường siêu dữ liệu kiểm toán, cấu trúc phân đoạn và tạo các bản sao JSON phục vụ đối soát, hoàn toàn nằm trong mục tiêu tối ưu hóa của hệ thống.

6 Bộ Điều Phối Tác Vụ và Quản Trị Cấu Hình

6.1 Cơ chế điều khiển luồng công việc qua run.py và PipelineDAG

Đường ống cung cấp giao diện dòng lệnh hợp nhất thông qua tệp tin trung tâm `run.py`, đóng vai trò là điểm nhập lệnh (*entry point*) cho toàn bộ các chế độ vận hành: từ thực thi đơn lẻ từng bước nghiệp vụ (`python run.py 1..7`), chạy chuỗi liên hoàn (`python run.py all`), kích hoạt tiến trình chạy ngầm định kỳ (`python run.py schedule`), cho tới chế độ điều phối cao cấp thông qua đồ thị có hướng không chu trình (`python run.py dag`).

Trong chế độ điều phối DAG (`src/orchestration/dag.py`), hệ thống tự động khởi tạo đối tượng `PipelineDAG` và liên kết 6 tác vụ tương ứng với 6 giai đoạn xử lý dữ liệu. Trình điều phối giải quyết các mối quan hệ phụ thuộc cha-con (`upstream_ids`) và tiến hành thực thi tuần tự theo thứ tự tô-pô (*topological execution*).



Hình 3: Đồ thị DAG thực thi tuần tự các tác vụ kèm chốt kiểm soát chất lượng (Quality Gates).

Một đặc tính quan trọng của `PipelineDAG` là cơ chế ngắt mạch sớm (*short-circuiting*). Nếu tác vụ số 3 (`step_3_ingest_filings`) xác nhận không có bất kỳ báo cáo 10-K mới nào trên máy chủ SEC và các tệp dữ liệu phái sinh của các tầng trước đã tồn tại đầy đủ, bộ điều phối lập tức đánh dấu các tác vụ 4, 5 và 6 sang trạng thái bỏ qua (`SKIPPED`), hoàn tất chu trình kiểm tra trạng thái chỉ trong 3,09 giây mà không tiêu tốn tài nguyên tính toán.

6.2 Chiến lược quản trị cấu hình và nhật ký kiểm toán có cấu trúc

Dự án quản trị cấu hình bằng mã nguồn Python chuẩn hóa kết hợp biến môi trường và tệp nhật ký có cấu trúc theo ba thành phần trọng tâm:

- **Quản trị đường dẫn vật lý (Path Management)**: Tệp `src/common/config.py` sử dụng lớp `pathlib.Path` thiết lập đường dẫn tuyệt đối cho các tầng Medallion (`RAW_DIR`, `STAGING_DIR`, `PRIMARY_DIR`, `CURATED_DIR`) từ gốc dự án, bảo đảm tính tương thích trên đa nền tảng điều hành (Windows/Linux/macOS).
- **Bảo mật và cấu hình môi trường (.env)**: Các thông tin nhạy cảm và định danh mô hình được lưu trữ độc lập trong tệp `.env` (`GEMINI_API_KEY`, mô hình nhúng `all-MiniLM-L6-v2` và mô hình tổng hợp `gemini-2.5-flash`), được nạp tự động qua thư viện `python-dotenv`.
- **Nhật ký kiểm toán có cấu trúc (Structured Audit Logging)**: Tệp `logs/orchestration_audit.jsonl` ghi nhận toàn bộ vết thực thi theo định dạng JSON Lines với các trường chuẩn: `run_id`, `dag_id`, `timestamp`, `task_id`, `status`, `duration_ms`, `retry_count`, `records_processed`, và `error_message`.

6.3 Đối sánh chuyên sâu với nền tảng Apache Airflow

Bảng 3 đối sánh các điểm tương đồng và khoảng trống kỹ thuật so với Apache Airflow:

Bảng 3: So sánh giữa bộ điều phối PipelineDAG nội bộ và Apache Airflow

Khía Cạnh Kỹ Thuật	Động Cơ PipelineDAG Cục Bộ	Apache Airflow Doanh Nghiệp
Cấu trúc đồ thị (DAG)	Quản lý phụ thuộc tô-pô đơn luồng, trạng thái rõ ràng	Hỗ trợ DAG phức tạp, đa nhánh, chạy phân tán
Xử lý lỗi & Thử lại	Tự động thử lại với Exponential Backoff & Full Jitter	Cấu hình retry, retry_delay, trigger_rule phong phú
Chốt chất lượng (Gate)	Tích hợp `pre_gate` và `post_gate` kiểm tra schema/size	Sử dụng `SQLCheckOperator`, DataHub sensor
Bộ thực thi (Executor)	Đơn tiến trình tuần tự cục bộ (Single-node runner)	CeleryExecutor, KubernetesExecutor phân tán tải
Hàng đợi thông điệp	Không dùng (gọi hàm Python trực tiếp trong bộ nhớ)	Quản lý hàng đợi phân tán qua RabbitMQ/Redis
Giao diện giám sát	Nhật ký văn bản và tệp JSON Lines tại `logs/`	Webserver UI xem sơ đồ cây, Gantt chart, live log
Chạy bù lịch sử	Chạy thủ công từng bước với tham số `-force`	Lệnh CLI backfill tự động theo khoảng thời gian

Thiết kế tinh gọn này giúp đường ống khởi động ngay tức thì trong vòng dưới 1 giây mà không chịu chi phí vận hành (*overhead*) cồng kềnh của một cụm máy chủ Airflow hoàn chỉnh, hoàn toàn phù hợp cho các luồng xử lý dữ liệu tài chính định kỳ theo lịch trình.

7 Tính Bất Biến (Idempotency) và Khả Năng Tái Lập

7.1 Nguyên lý bất biến và cơ chế ghi đè có kiểm soát

Trong kỹ thuật dữ liệu chuyên nghiệp, tính bất biến (*idempotency*) là nguyên tắc bắt buộc: việc thực thi một đường ống nhiều lần với cùng một tập dữ liệu đầu vào phải luôn tạo ra cùng một trạng thái đích duy nhất, tuyệt đối không làm phát sinh dữ liệu dư thừa, sai lệch hoặc trùng lặp bản ghi.

Đường ống đạt được tính bất biến thông qua cơ chế ghi đè có kiểm soát (*Idempotent Overwrite Pattern*) trên toàn bộ các tầng dữ liệu:

- **Đường dẫn vật lý tất định (Deterministic File Paths)**: Định tuyến mỗi thực thể dữ liệu tới một đường dẫn tệp tin cố định dựa trên các thuộc tính tự nhiên của dữ liệu (ví dụ: `data/02_staging/sec_filings/NVDA/NVDA_2026_risk_factors.md` hoặc `data/03_primary/chunks/all_chunks.parquet`), kiên quyết không dùng chuỗi thời gian ngẫu nhiên.
- **Cơ chế ghi tệp nguyên tử có thay thế (Atomic Overwrite)**: Xuất thẳng DataFrame ra đĩa thông qua các hàm ghi đè hoàn toàn (`to_parquet` với chế độ mặc định, `write_text` ghi đè toàn bộ tệp). Hệ thống không áp dụng cơ chế nối tiếp dữ liệu (`append mode`). Dù đường ống chạy lại 1 lần hay 100 lần, bảng `all_chunks.parquet` vẫn duy trì chính xác 925 hàng.
- **Đồng bộ Tầng Phục vụ ChromaDB**: Khi nạp dữ liệu vào cơ sở dữ liệu véc-tơ, hàm `build_vector_index` thực hiện truy xuất bộ sưu tập `sec_risk_factors` và cập nhật theo danh sách định danh hoặc tái tạo bộ sưu tập mới nếu cấu trúc dữ liệu bị thay đổi, ngăn chặn việc phát sinh các bản ghi véc-tơ mồ côi (*orphaned vectors*).

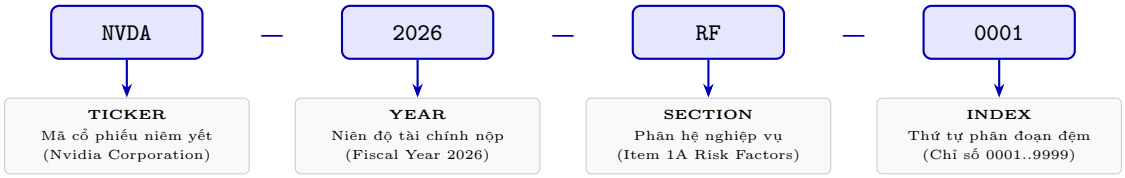
7.2 Định danh phân đoạn tất định và kiểm toán nguồn gốc

Khác với các ứng dụng thông thường sử dụng chuỗi định danh ngẫu nhiên (như UUID v4) để đánh dấu các đoạn văn bản dẫn đến sự xáo trộn mã khóa qua mỗi lần chạy lại, đường ống thiết lập quy tắc tạo mã định danh duy nhất tất định (*Deterministic Identifier*) cho từng phân đoạn văn bản tại Tầng Vàng:

$$\mathrm{ID}_{\mathrm{chunk}} = \mathrm{TICKER} _ \mathrm{YEAR} _ \mathrm{RF} _ \mathrm{INDEX}_{04d} \tag{3}$$

Ví dụ cụ thể: đoạn phân đoạn đầu tiên của tập đoàn Nvidia trong niên độ tài chính 2026 luôn có mã số cố định là `NVDA_2026_RF_0001`. Quy tắc định danh này đem lại ba lợi ích kỹ thuật thiết yếu:

- **Khả năng so sánh sai khác (Deterministic Diffing)**: Cho phép so sánh trực tiếp giữa các lần chạy lại để phát hiện chính xác phân đoạn nào có sự biến động nội dung khi tài liệu gốc thay đổi.
- **Truy vết nguồn gốc kiểm toán 100% (Full Data Provenance)**: Cho phép truy nguyên nguồn gốc từ bất kỳ kết quả truy hồi nào quay ngược trở lại đúng tệp tin Markdown tại Tầng Bạc và tệp HTML gốc tại Tầng Đồng.



Hình 4: Sơ đồ phân rã cấu trúc định danh phân đoạn tất định (Deterministic Chunk ID Schema).

– **Tối ưu hóa bộ nhớ đệm (Safe Embedding Cache)**: Giúp các hệ thống bộ nhớ đệm ở tầng ứng dụng hạ nguồn tái sử dụng các véc-tơ nhúng đã tính toán trước đó một cách an toàn.

7.3 Tính cô lập bước và quy trình tái lập hệ thống

Mỗi bước xử lý trong đường ống được thiết kế theo nguyên lý ghép nối lỏng (*loose coupling*) và cô lập trạng thái (*step isolation*). Dữ liệu đầu vào của Bước N phụ thuộc hoàn toàn vào tệp tin vật lý được tạo ra bởi Bước $N-1$. Do đó, nếu tệp tin của Tầng Bạc đã tồn tại, kỹ sư có thể kích hoạt riêng rẽ Bước 4 hoặc Bước 5 (`python run.py 4 NVDA` hoặc `python run.py 5`) để thử nghiệm các giải thuật bóc tách mới mà không cần chạy lại các bước mạng phía trước. Nếu phát hiện tệp tin tiền đề chưa sẵn sàng, hàm nghiệp vụ sẽ đưa ra cảnh báo `FileNotFoundError` rõ ràng.

Quy trình tái lập toàn bộ trạng thái dữ liệu từ mã nguồn được tiêu chuẩn hóa theo 4 thao tác tuần tự:

1. **Nhân bản mã nguồn**: Tải mã nguồn từ kho lưu trữ GitHub: `https://github.com/Accnoname/sec-financial-data-pipeline.git`.
2. **Khởi tạo môi trường ảo**: Thiết lập môi trường Python 3.10+ (`python -m venv .venv`).
3. **Cài đặt gói phụ thuộc**: Cài đặt chính xác các thư viện qua `pip install -r requirements.txt`.
4. **Thực thi điều phối cưỡng bức và kiểm thử**: Chạy toàn bộ chu trình với `python run.py dag -force` kết hợp bộ kiểm tra `pytest tests/ -v`.

Toàn bộ 4 tầng hồ chứa dữ liệu sẽ được tái lập trọn vẹn trong khoảng 40 giây và vượt qua 100% các kiểm thử chất lượng dữ liệu.

8 Quản Trị Chất Lượng Dữ Liệu và Bộ Kiểm Thử Tự Động

8.1 Triết lý kiểm thử theo hướng hợp đồng dữ liệu (Data Contract Testing)

Trong các hệ thống phân tích hiện đại, kiểm thử dữ liệu không chỉ đơn thuần là kiểm tra tính đúng đắn của mã lệnh phần mềm (*software logic testing*) mà quan trọng hơn là kiểm thử các giả định về dữ liệu (*data assumptions testing*). Bất kỳ một biến động ngầm nào trong cấu trúc siêu văn bản của SEC hay sự thay đổi kiểu dữ liệu đều có thể dẫn đến sự sập đổ dây chuyền của các mô hình phân tích hạ nguồn.

Dự án thiết lập một hệ thống bảo vệ chất lượng toàn diện gồm 24 ca kiểm thử tự động hóa bằng công cụ Pytest, phân bổ thành hai tầng:

– **Tầng kiểm thử đơn vị độc lập (`tests/unit/`, 16 ca test)**: Kiểm tra tính đúng đắn của các hàm chuẩn hóa CIK, giải thuật phân đoạn văn bản, cơ chế thử lại của DAG và công thức xếp hạng.

– **Tầng kiểm thử tích hợp hợp đồng dữ liệu (`tests/test_pipeline.py`, 8 ca test)**: Kiểm soát tính toàn vẹn vật lý, độ dài tệp tin và lược đồ dữ liệu qua từng tầng Medallion.

8.2 Phân tích chuyên sâu 8 ca kiểm thử tích hợp đường ống

Bảng 4 liệt kê chi tiết 8 ca kiểm thử tích hợp cốt lõi trong `test_pipeline.py`, phân định rạch ròi những gì bài kiểm tra đang bảo đảm và những gì bài kiểm tra chưa bao phủ:

8.3 Hợp đồng dữ liệu ràng buộc độ dài véc-tơ nhúng

Đặc biệt, ca kiểm thử số 7 (`test_gold_chunks_fit_embedding_context_window`) đóng vai trò là một cổng kiểm soát chất lượng (Quality Gate) tiên quyết. Bài kiểm tra khởi tạo chính xác bộ tách từ của mô hình `all-MiniLM-L6-v2` và duyệt qua toàn bộ 925 hàng trong tệp `all_chunks.parquet`:

$$\forall c \in \text{GoldChunks}, \quad \text{len}(\text{Tokenizer.encode}(c.\text{text})) \leq 256 \tag{4}$$

Bảng 4: Phân tích phạm vi bảo đảm và giới hạn của 8 ca kiểm thử tích hợp trong test_pipeline.py

Tên Ca Kiểm Thử	Được Bảo Đảm (Verified)	Chưa Bảo Đảm (Omitted)
`test_raw_companies_json_exists`	Tệp thô `all_companies.json` tồn tại, size $>$ 100 KB.	Chưa kiểm tra schema sâu của từng phần tử JSON.
`test_raw_sec_filings_exist`	Đủ 7 thư mục, mỗi tệp HTML Form 10-K gốc $>$ 500 KB.	Chưa kiểm tra tính toàn vẹn cú pháp HTML nội dung.
`test_staging_parquet_schema`	Tồn tại Parquet Silver, đủ 5 cột, mẫu 50 CIK đủ 10 số.	Chưa xác thực chéo CIK với cơ sở dữ liệu online SEC.
`test_markdown_risk_factors`	Đủ 7 tệp Markdown Item 1A, chứa từ khóa rủi ro $>$ 10 KB.	Chưa kiểm tra độ tinh khiết ngữ nghĩa ranh giới Item 1B.
`test_gold_chunks_schema_content`	Tồn tại Parquet Gold, 8 cột, 7 mã, text không rỗng.	Chưa đo mức độ trùng lặp nội dung sliding window.
`test_curated_vector_db`	Kết nối ChromaDB thành công, collection có $\geq$900 vector.	Chưa đánh giá độ chính xác toán học của vector nhúng.
`test_gold_chunks_fit_embedding`	**Hợp đồng dữ liệu:** 100% của 925 chunk $\leq$ 256 tokens.	Chưa đo lường độ trễ suy diễn (latency) trên GPU/CPU.
`test_rag_prompt_construction`	Khuôn mẫu prompt ghép đúng nguồn `[Source 1]` & query.	Chưa kiểm tra rủi ro tấn công prompt injection.

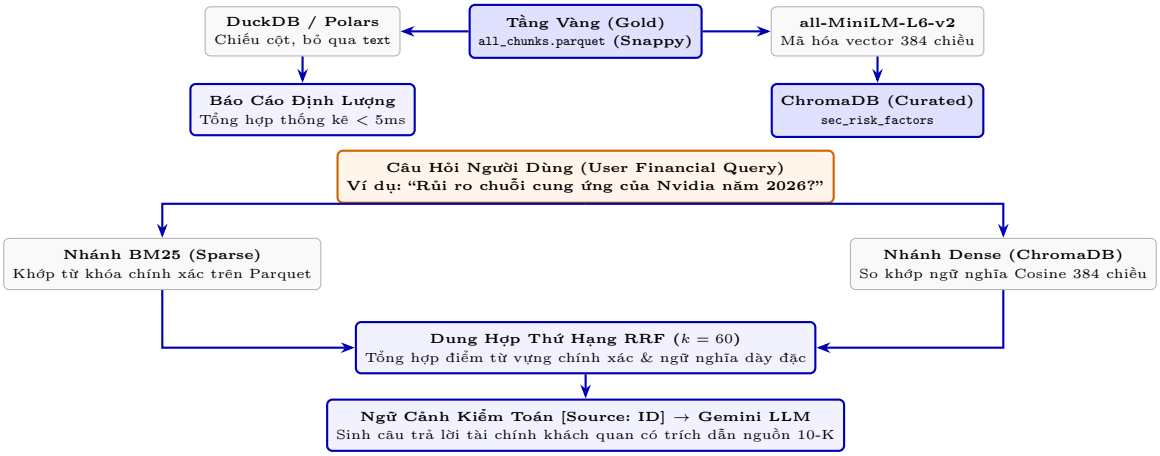
Nếu phát hiện bất kỳ phân đoạn nào vi phạm giới hạn 256 tokens, quá trình kiểm thử sẽ lập tức kích hoạt lỗi khẳng định `AssertionError` và hiển thị danh sách các phân đoạn vi phạm, ngăn chặn việc đẩy dữ liệu lỗi lên Tầng Phục vụ. Toàn bộ 24 ca kiểm thử trong hệ thống thực thi hoàn tất trong 27,5 giây, tạo nên một lưới an toàn kỹ thuật tin cậy cho toàn bộ đường ống.

9 Tầng Phục Vụ (Serving Layer)

9.1 Cơ chế nạp và lập chỉ mục tại Tầng Phục Vụ

Sau khi dữ liệu phân đoạn được chuẩn hóa toàn diện tại Tầng Vàng (`all_chunks.parquet`), đường ống tiến hành bàn giao dữ liệu sang Tầng Phục Vụ (Curated Layer) thông qua mô-đun `src/rag/embedder.py`. Hệ thống triển khai giải pháp cơ sở dữ liệu véc-tơ cục bộ ChromaDB (`PersistentClient`), lưu trữ trực tiếp dưới dạng các tệp tin nhị phân và SQLite tại thư mục `data/04_curated/vector_db/`.

Quá trình chuyển đổi véc-tơ hóa sử dụng mô hình mã hóa ngôn ngữ nhỏ gọn `all-MiniLM-L6-v2` [3], ánh xạ từng đoạn văn bản thành một véc-tơ nhúng dày đặc 384 chiều trong không gian Euclid. Toàn bộ 925 đoạn văn bản cùng các trường siêu dữ liệu kiểm toán (`chunk_id`, `ticker`, `year`, `section`, `chunk_index`) được nạp vào bộ sưu tập mang tên `sec_risk_factors` với độ đo khoảng cách Cosine chuẩn hóa.



Hình 5: Sơ đồ luồng truy xuất dữ liệu hai kênh: Kênh phân tích định lượng trực tiếp từ Tầng Vàng và Kênh tìm kiếm kết hợp Hybrid RAG phục vụ người dùng.

9.2 Khai thác dữ liệu qua động cơ tìm kiếm kết hợp Hybrid Search

Dữ liệu ra khỏi đường ống được khai thác trực tiếp bởi động cơ truy hồi thông tin kết hợp (`src/rag/hybrid_retriever.py`), kết hợp hài hòa giữa khả năng hiểu ngữ nghĩa sâu và độ chính xác từ khóa rời rạc:

– **Nhánh truy hồi ngữ nghĩa dày đặc (Dense Retrieval)**: Mã hóa câu hỏi của người dùng thành véc-tơ 384 chiều và so khớp với 925 véc-tơ trong ChromaDB để tìm ra top-k phân đoạn có độ tương đồng Cosine cao nhất, hỗ trợ lọc theo mã cổ phiếu và năm tài chính.

– **Nhánh tìm kiếm từ vựng rời rạc (Sparse BM25 Path)**: Sử dụng bộ tách từ loại bỏ từ dừng và tính toán điểm số khớp từ vựng chính xác dựa trên tần suất từ (TF) và nghịch đảo tần suất văn bản (IDF) theo thuật toán BM25 Okapi [4] đọc trực tiếp từ tệp Parquet Tầng Vàng.

– **Dung hợp thứ hạng nghịch đảo (Reciprocal Rank Fusion – RRF)**: Kết quả từ hai nhánh được hợp nhất phi tham số với hằng số làm mượt $k = 60$ [5]:

$$\text{Score}_{\text{RRF}}(d) = \frac{1}{60 + \text{Rank}_{\text{Dense}}(d)} + \frac{1}{60 + \text{Rank}_{\text{BM25}}(d)} \tag{5}$$

9.3 Cung cấp ngữ cảnh kiểm toán cho mô hình ngôn ngữ lớn

Đầu ra cuối cùng của Tầng Phục Vụ được định tuyến tới mô-đun `src/rag/retriever.py`. Các phân đoạn đạt điểm RRF cao nhất được tổ chức thành một khối ngữ cảnh có cấu trúc kèm nhãn trích dẫn tường minh (ví dụ: `[Source 1: NVDA 2026, Item 1A, chunk 1]`). Mô hình ngôn ngữ lớn (Gemini AI) tiếp nhận ngữ cảnh này cùng câu hỏi của chuyên viên tài chính để tổng hợp câu trả lời khách quan, cam kết không bịa đặt thông tin và cung cấp đường dẫn truy nguyên ngược về đúng báo cáo Form 10-K gốc.

10 Ca Nghiên Cứu: Khắc Phục Lỗi Cắt Ngắn Ngữ Cảnh (Truncation Defect)

10.1 Bản chất của lỗi suy giảm chất lượng dữ liệu âm thầm

Trong kỹ thuật dữ liệu phục vụ học máy và AI, lỗi nguy hiểm nhất không phải là các lỗi cú pháp gây sập chương trình (*runtime crashes*), mà là các lỗi chất lượng dữ liệu âm thầm (*silent data quality defects*) – nơi đường ống vẫn thực thi hoàn tất thành công nhưng dữ liệu bị suy hao giá trị thông tin mà không phát ra bất kỳ cảnh báo nào.

Trong phiên bản sơ khởi của hệ thống, thuật toán phân đoạn văn bản sử dụng phương pháp đếm từ truyền thống: mỗi đoạn chứa 500 từ tiếng Anh và trượt gối đầu 100 từ, sinh ra tổng cộng 262 phân đoạn cho 7 báo cáo Form 10-K. Tuy nhiên, một từ tiếng Anh trong các tài liệu tài chính phức tạp thường tương đương từ 1,3 đến 1,8 tokens sau khi qua bộ tách từ WordPiece/BPE. Khi tiến hành kiểm định độ dài thực tế bằng bộ tách từ của mô hình `all-MiniLM-L6-v2`, đường ống phát hiện một thực trạng nghiêm trọng: có tới **257 trong tổng số 262 phân đoạn (chiếm 98,1%)** có độ dài vượt quá giới hạn tối đa 256 tokens của mô hình nhúng véc-tơ.

Mô hình học máy xử lý tình huống này bằng cách âm thầm cắt bỏ (*truncate*) toàn bộ các tokens từ vị trí 257 trở đi trong quá trình tính toán véc-tơ. Hậu quả là hơn 50% nội dung văn bản ở nửa sau của từng phân đoạn hoàn toàn biến mất khỏi không gian véc-tơ nhúng, tạo ra các vùng "điểm mù ngữ cảnh" khổng lồ tại Tầng Phục Vụ.

10.2 Thiết kế thực nghiệm đo lường và phát hiện điểm mù

Để đo lường định lượng mức độ thiệt hại của lỗi cắt ngắn ngữ cảnh, dự án xây dựng một bộ dữ liệu kiểm định chuẩn mực (`benchmarks/eval_questions.json`) gồm 20 câu hỏi nghiệp vụ bao quát các rủi ro cốt lõi của 7 tập đoàn công nghệ. Nhóm câu hỏi được phân bổ có chủ đích thành hai nhóm đối xứng: 10 câu hỏi có văn bản neo (*anchor text*) nằm ở nửa đầu phân đoạn cũ (`late_in_old_chunk: false`, độ sâu dưới 250 tokens) và 10 câu hỏi có văn bản neo nằm sâu ở nửa sau phân đoạn cũ (`late_in_old_chunk: true`, độ sâu vượt quá 256 tokens).

Hệ thống đo lường hiệu năng truy xuất thông qua chỉ số Tỷ lệ tìm trúng (Hit@k) và Thứ hạng nghịch đảo trung bình (Mean Reciprocal Rank – MRR), đối sánh với đường cơ sở ngẫu nhiên Monte Carlo qua 500 vòng lặp.

10.3 Giải pháp kỹ thuật và bằng chứng thực nghiệm phục hồi dữ liệu

Đường ống tiến hành tái cấu trúc toàn diện giải thuật phân đoạn tại Tầng Vàng (`src/rag/chunking.py`): chuyển đổi từ bộ đếm từ thô sơ sang giải thuật phân tách nhận biết bộ tách từ (*tokenizer-aware chunking*). Giải thuật tôn trọng ranh giới kết thúc câu, thiết lập kích thước mục tiêu 210 tokens, bước trượt 40 tokens và áp đặt một trần cứng tuyệt đối là 240 tokens. Toàn bộ tập dữ liệu được tái cấu trúc thành 925 phân đoạn đồng đều, được bảo vệ bởi hợp đồng dữ liệu tự động `test_gold_chunks_fit_embedding_context_window` trong Pytest.

Bảng 5 cung cấp bằng chứng thực nghiệm rõ ràng về hiệu quả của việc sửa chữa lỗi dữ liệu từ gốc:

– **Phục hồi vượt bậc tại điểm mù véc-tơ (+50%)**: Tỷ lệ tìm kiếm chính xác (Hit@15) của mô hình Dense trên nhóm câu hỏi bị rơi vào điểm mù cũ đã tăng vọt từ 6/10 lên 9/10 (tương ứng mức phục hồi 50%), đưa hiệu năng tổng thể của mô hình Dense đạt 18/20 (90%), ngang bằng với hiệu năng cao nhất của BM25.

Bảng 5: Bằng chứng thực nghiệm đối đầu trước và sau khi khắc phục lỗi cắt ngắn ngữ cảnh

Cấu Hình & Phân Vùng Câu Hỏi	Ngẫu Nhiên	Dense (MiniLM)	BM25 Okapi	Hybrid (RRF)	Tốt Nhất
1. Cấu hình cũ bị lỗi (Phân đoạn 500 từ, 262 chunks, 98,1% vượt trần context)					
Toàn bộ câu hỏi Hit@5 (∼3.000t)	3,6 / 20	14 / 20	**18 / 20**	17 / 20	**18 / 20** (BM25)
Nhóm nửa sau (Điểm mù) Hit@5	—	6 / 10	**9 / 10**	8 / 10	**9 / 10** (BM25)
Nhóm nửa đầu (Bình thường) Hit@5	—	8 / 10	**9 / 10**	**9 / 10**	**9 / 10** (Đồng hạng)
Chỉ số MRR	0,0849	0,5375	**0,6975**	0,6417	**0,6975** (BM25)
2. Cấu hình mới đã sửa (Token-aware ≤240t, 925 chunks, 0% vượt trần context)					
Toàn bộ câu hỏi Hit@15 (∼3.000t)	3,1 / 20	**18 / 20**	16 / 20	**18 / 20**	**18 / 20** (Dense/Hyb)
Nhóm nửa sau (Điểm mù cũ) Hit@15	—	**9 / 10**	7 / 10	**9 / 10**	**9 / 10 (+50%)**
Nhóm nửa đầu (Bình thường) Hit@15	—	9 / 10	9 / 10	9 / 10	9 / 10 (Đồng hạng)
Chỉ số MRR	0,0337	0,4010	**0,4326**	0,4297	**0,4326** (BM25)

- **Đánh đổi về kích thước ngữ cảnh của BM25**: Ngược lại, thuật toán BM25 bị giảm nhẹ từ 18/20 xuống 16/20 (-2 câu hỏi) do các phân đoạn ngắn hơn làm mất đi một số từ khóa ngữ cảnh mở rộng xung quanh.

Ca nghiên cứu khẳng định một nguyên lý cốt lõi trong kỹ thuật dữ liệu: chất lượng và sự am hiểu tường tận về các đặc tính biểu diễn của dữ liệu quyết định giới hạn trên của toàn bộ các hệ thống AI phân tích hạ nguồn.

11 Vận Hành Thực Tế và Khả Năng Mở Rộng

11.1 Thời gian thực thi từng bước trong môi trường vận hành

Hiệu năng thời gian thực thi của đường ống được thu thập và định lượng thông qua tệp nhật ký kiểm toán có cấu trúc `logs/orchestration_audit.jsonl`. Bảng 6 phản ánh chi tiết thời gian chạy trong hai kịch bản điển hình: kịch bản khởi động nguội toàn phần (Cold Run – nạp mới toàn bộ dữ liệu từ đầu) và kịch bản chạy cảm biến gia tăng định kỳ (Incremental Sensor Run):

Bảng 6: Đo lường thời gian thực thi chi tiết của từng bước trong đường ống

Bước	Tên Tác Vụ Nghiệp Vụ	Cold Run	Incremental Run	Ghi Chú Trạng Thái
Bước 1	`step_1_extract_companies`	3.207 ms	0,5 ms	Bỏ qua do tệp > 100 KB
Bước 2	`step_2_clean_companies`	121 ms	128 ms	Xử lý 10.400 bản ghi sang Parquet
Bước 3	`step_3_ingest_filings`	∼15.000 ms	3.093 ms	Kiểm tra 7 CIK submission API
Bước 4	`step_4_extract_risk_factors`	5.803 ms	0,0 ms	`SKIPPED` (Cảm biến ngắt sớm)
Bước 5	`step_5_chunk_risk_factors`	145 ms	0,0 ms	`SKIPPED` (Cảm biến ngắt sớm)
Bước 6	`step_6_build_vector_index`	16.693 ms	0,0 ms	`SKIPPED` (Cảm biến ngắt sớm)
Tổng thời gian thực thi toàn chu trình		**∼41.000 ms**	**3.221 ms**	**Nhanh hơn 12,7 lần**

Số liệu chứng minh cơ chế cảm biến chênh lệch mang lại lợi ích vận hành to lớn: trong chế độ giám sát tự động hàng ngày (`setup_windows_task.bat` chạy lúc 02:00 AM), nếu không có hồ sơ Form 10-K mới nào được nộp lên SEC, toàn bộ đường ống hoàn thành trong vỏn vẹn **3,22 giây**, tiết kiệm 100% tài nguyên CPU cho các tác vụ bóc tách và nhúng véc-tơ.

11.2 Phân tích các điểm nghẽn kiến trúc khi dữ liệu tăng gấp 100 lần

Nếu phạm vi của đường ống được mở rộng gấp 100 lần (từ 7 công ty lên 700 tập đoàn lớn, tương đương xử lý 700 tệp báo cáo Form 10-K với dung lượng thô khoảng 3 GB đến 5 GB và sinh ra gần 100.000 phân đoạn văn bản), hệ thống kiến trúc đơn máy hiện tại sẽ bộc lộ bốn điểm nghẽn kỹ thuật rõ rệt:

- **Điểm nghẽn mạng thu thập (Network Ingestion Bottleneck)**:
  - *Thách thức*: Do trần giới hạn tốc độ 10 req/s của SEC, tải tuần tự 700 tệp HTML kèm 700 cuộc gọi siêu dữ liệu sẽ kéo dài 15–20 phút. Mở rộng đa luồng không kiểm soát sẽ bị chặn mã HTTP 429 hoặc khóa địa chỉ IP.
  - *Giải pháp*: Thiết lập bộ điều tiết tốc độ phân tán (*Distributed Token Bucket Rate Limiter*) kết hợp mạng lưới máy chủ ủy thác xoay vòng (*Proxy Pool*) tuân thủ quy chuẩn SEC.
- **Điểm nghẽn bộ nhớ và CPU khi phân tích DOM (Parsing Bottleneck)**:
  - *Thách thức*: Thư viện `BeautifulSoup` nạp toàn bộ cây DOM vào RAM. Xử lý đồng thời 700 tệp phức tạp trên tiến trình đơn luồng sẽ dẫn đến lỗi cạn kiệt bộ nhớ (OOM) và tiêu tốn hơn 10 phút CPU.
  - *Giải pháp*: Chuyển đổi sang kiến trúc đa tiến trình song song (`multiprocessing`) hoặc áp dụng trình phân tích luồng sự kiện SAX (*Streaming XML/HTML Parser*) để đọc tệp theo luồng mà không giữ DOM trong

RAM.

- **Điểm nghẽn lưu trữ tệp đơn (Storage Bottleneck)**:
  - *Thách thức*: Hợp nhất gần 100.000 phân đoạn vào duy nhất một tệp `all_chunks.parquet` gây nghẽn I/O và làm chậm truy cập đồng thời.
  - *Giải pháp*: Áp dụng phân vùng dữ liệu hồ chứa (*Hive-style Partitioning*) theo thư mục `year=YYYY/ticker=XXXX/` để tận dụng kỹ thuật cắt tỉa phân vùng (*Partition Pruning*).
- **Điểm nghẽn cơ sở dữ liệu véc-tơ (Vector Indexing Bottleneck)**:
  - *Thách thức*: ChromaDB nhúng trên SQLite cục bộ gặp hiện tượng xung đột khóa tệp (*file lock contention*) và suy giảm hiệu năng khi vượt ngưỡng 100.000 véc-tơ; tính toán trên CPU mất gần 30 phút.
  - *Giải pháp*: Chuyển dịch Tầng Phục Vụ sang cụm cơ sở dữ liệu véc-tơ chuyên dụng (Qdrant, Milvus hoặc `pgvector`) kết hợp tăng tốc tính toán nhúng theo lô trên vi xử lý đồ họa GPU.

12 Giới Hạn Kỹ Thuật và Hướng Phát Triển

12.1 Các hạn chế cố hữu trong phiên bản hiện tại

Mặc dù đường ống đã giải quyết thành công các bài toán cốt lõi về tính bất biến, tối ưu hóa lưu trữ và khắc phục lỗi suy giảm chất lượng dữ liệu, hệ thống vẫn tồn tại ba hạn chế kỹ thuật chính cần được ghi nhận minh bạch:

- **Điều phối đơn tiến trình cục bộ**: Động cơ PipelineDAG nội bộ chạy trực tiếp trên một tiến trình Python duy nhất. Việc thiếu một bộ lập lịch phân tán chuẩn mực (như Celery Executor hay Kubernetes Pod Operator) giới hạn khả năng mở rộng quy mô khi cần thu thập đồng thời hàng nghìn báo cáo trong các tuần cao điểm nộp hồ sơ.
- **Cơ chế cảm biến ở cấp độ tệp tin (File-level Delta)**: Hệ thống phát hiện thay đổi dựa trên mã hồ sơ nộp `accessionNumber`. Khi có báo cáo mới, toàn bộ Mục 1A của công ty đó được trích xuất và tính toán véc-tơ hóa lại từ đầu, chưa hỗ trợ so khớp thay đổi ở cấp độ từng câu văn bản (*Semantic Sentence Diff*) để chỉ nhúng lại phần nội dung mới.
- **Thiếu cơ chế cảnh báo thời gian thực (Real-time Observability)**: Việc giám sát phụ thuộc chủ yếu vào tệp nhật ký `orchestration_audit.jsonl` và dòng lệnh, chưa có bảng điều khiển trực quan tập trung (Dashboard) để gửi cảnh báo tức thì qua email hoặc webhook khi tác vụ bị treo hoặc tỷ lệ lỗi mạng tăng đột biến.

12.2 Lộ trình nâng cấp và mở rộng kiến trúc tương lai

Để nâng cấp đường ống từ một hệ thống độc lập thành một nền tảng dữ liệu tài chính cấp doanh nghiệp (*Enterprise Financial Data Platform*), các định hướng nghiên cứu và phát triển tiếp theo tập trung vào ba trụ cột chiến lược:

- **Đóng gói vi dịch vụ và điều phối với Apache Airflow**: Chuyển đổi từng bước nghiệp vụ (`extract`, `clean`, `chunk`, `embed`) thành các container Docker độc lập, thiết lập luồng điều phối trên Apache Airflow hoặc Prefect triển khai trên Kubernetes, tận dụng khả năng tự động co giãn (*Autoscaling*) và tích hợp cảnh báo qua Slack/Teams.
- **Tích hợp định dạng bảng hồ chứa mở Apache Iceberg**: Nâng cấp bảng Parquet lên định dạng bảng hồ chứa mở Apache Iceberg hoặc Delta Lake nhằm hỗ trợ giao dịch ACID, cho phép truy vấn dữ liệu lịch sử theo mốc thời gian (*Time Travel*) để kiểm toán phiên bản rủi ro qua từng năm, và tự động tối ưu hóa kích thước tệp nén trên đám mây (AWS S3 hoặc Google Cloud Storage).
- **Hệ thống giám sát trôi dạt dữ liệu (Data Drift Monitoring)**: Tích hợp nền tảng kiểm định chất lượng tự động Great Expectations vào các chốt kiểm soát (Quality Gates), kết hợp Evidently AI để theo dõi sự biến động phân phối từ khóa và phát hiện hiện tượng trôi dạt ngữ nghĩa (*Concept Drift*) của các yếu tố rủi ro kinh doanh theo thời gian thực.

Tài liệu

[1] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics," *Proceedings of CIDR*, vol. 2021, p. 8, 2021.

[2] U.S. Securities and Exchange Commission, "Accessing edgar data: Webmaster best practices and automated ingestion guidelines," https://www.sec.gov/os/accessing-edgar-data, 2024.

[3] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[4] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends® in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[5] G. V. Cormack, C. L. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual run learning methods," *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pp. 758–759, 2009.

Phụ Lục: Bảng Chú Giải Thuật Ngữ Kỹ Thuật Chuyên Ngành

Nhằm hỗ trợ việc tra cứu, chuẩn hóa khái niệm và phục vụ công tác thẩm định kỹ thuật độc lập, Bảng 7 cung cấp định nghĩa chi tiết của các thuật ngữ cốt lõi trong kỹ thuật dữ liệu (Data Engineering) và hệ thống trí tuệ nhân tạo (AI/RAG) được sử dụng xuyên suốt công trình:

Bảng 7: Bảng chú giải các thuật ngữ kỹ thuật chuyên ngành cốt lõi trong đường ống

Thuật Ngữ Kỹ Thuật	Định Nghĩa Bản Chất & Vai Trò Trong Đường Ống Dữ Liệu
SEC EDGAR	Hệ thống cơ sở dữ liệu điện tử chính thức của Ủy ban Chứng khoán Hoa Kỳ (Electronic Data Gathering, Analysis, and Retrieval) lưu trữ và phân phối hồ sơ tài chính công.
Form 10-K	Báo cáo thường niên toàn diện bắt buộc theo Luật Giao dịch Chứng khoán Hoa Kỳ năm 1934, phản ánh đầy đủ tình hình kinh doanh, tài chính và pháp lý của công ty đại chúng.
Item 1A (Risk Factors)	Phân mục bắt buộc trong Form 10-K, nơi doanh nghiệp công bố toàn bộ các rủi ro trọng yếu có thể đe dọa hoạt động kinh doanh, lợi nhuận hoặc giá trị cổ phiếu.
CIK (Central Index Key)	Mã số nhận diện pháp lý duy nhất gồm 10 chữ số do SEC cấp phát cho từng tổ chức hoặc cá nhân nộp hồ sơ, không thay đổi theo thời gian.
Accession Number	Mã số kiểm soát duy nhất gồm 20 ký tự (dạng `0000320193-25-000106`) gán cho mỗi lượt nộp hồ sơ cụ thể lên máy chủ SEC, gồm CIK, năm nộp và số thứ tự.
iXBRL (Inline XBRL)	Chuẩn báo cáo tài chính điện tử quốc tế tích hợp mã siêu dữ liệu XML máy đọc được trực tiếp vào bên trong các thẻ siêu văn bản HTML người đọc được.
Medallion Architecture	Mô hình thiết kế hồ lưu trữ dữ liệu (Lakehouse) tổ chức dữ liệu theo các tầng tinh chế lũy tiến gồm Bronze (thô), Silver (làm sạch), Gold (tính toán) và Curated (phục vụ).
Snappy Parquet	Định dạng tệp lưu trữ theo hướng cột chuẩn mở (Apache Parquet) kết hợp thuật toán nén Snappy, tối ưu hóa phép chiếu cột (Column Projection) và nén từ điển.
Idempotency (Tính bất biến)	Đặc tính của một tác vụ dữ liệu khi thực thi nhiều lần với cùng tập dữ liệu đầu vào luôn sinh ra cùng một trạng thái đích duy nhất, loại bỏ hoàn toàn nguy cơ nhân đôi dữ liệu.
Delta Sensor (Cảm biến)	Cơ chế thăm dò siêu dữ liệu nhẹ kiểm tra mã `accessionNumber` để phát hiện trạng thái thay đổi của tệp nguồn trước khi kích hoạt các tác vụ tải dữ liệu nặng.
Short-circuiting (Ngắt mạch)	Khả năng của bộ điều phối DAG tự động bỏ qua (skip) các tác vụ tính toán hạ nguồn khi cảm biến xác nhận không có bất kỳ dữ liệu mới nào phát sinh.
Quality Gate (Chốt chất lượng)	Các điểm kiểm soát logic tự động đặt giữa các tác vụ, tiến hành kiểm tra lược đồ, dung lượng và ràng buộc nghiệp vụ trước khi cho phép dữ liệu chuyển tầng.
Tokenizer-aware Chunking	Thuật toán phân đoạn văn bản tôn trọng cấu trúc từ vựng của bộ tách từ (Subword/WordPiece) và ranh giới câu, đảm bảo 100% các đoạn đều không vượt trần token.
Context Window Truncation	Lỗi suy giảm chất lượng dữ liệu âm thầm khi mô hình học sâu tự động cắt bỏ phần token vượt quá dung lượng đầu vào tối đa mà không phát ra bất kỳ cảnh báo lỗi nào.
BM25 Okapi	Thuật toán xếp hạng từ vựng kinh điển dựa trên mô hình xác suất, tính toán mức độ liên quan giữa câu truy vấn và văn bản dựa trên tần suất từ (TF) và nghịch đảo tần suất văn bản (IDF).
Dense Vector Retrieval	Phương pháp truy hồi ngữ nghĩa dày đặc, ánh xạ văn bản thành các véc-tơ nhiều chiều trong không gian Euclid và tính toán độ tương đồng thông qua khoảng cách Cosine.
Reciprocal Rank Fusion (RRF)	Thuật toán dung hợp thứ hạng phi tham số kết hợp kết quả từ nhiều động cơ tìm kiếm khác nhau dựa trên tổng nghịch giá thứ tự xếp hạng, triệt tiêu độ lệch điểm số thô.